

UNIVERSIDAD NACIONAL DE INGENIERÍA

# Facultad de Ciencias

## Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

**2026-1**

## Sesión 13:

# Clases

# Contenido

1. Introducción: ¿Por qué clases?
2. Anatomía de una clase
3. Constructores y ciclo de vida del objeto
4. El puntero this
5. Invariantes de clase y diseño defensivo

```
struct Circulo {
    double radio;
};

void setRadio(Circulo& c, double r) { c.radio = r; }
double getArea(Circulo& c) { return 3.14 * c.radio * c.radio; }
void   imprimir(Circulo& c) { cout << "Radio: " << c.radio << endl; }

int main() {
    Circulo c1;
    c1.radio = -50.0;    //  ¿el compilador se queja?


    imprimir(c1);
    return 0;
}
```

# Los tres problemas concretos

1. Cualquiera puede escribir c1.radio = -50; (dato público, no hay barrera)

2. Las funciones y el dato están desconectados
   setRadio, getArea e imprimir son funciones libres que podrían operar sobre cualquier struct, no necesariamente sobre Circulo.

3. No hay garantía de estado válido (un Circulo puede existir con radio = 0, radio = -5, o sin inicializar).

Hemos visto patrones similares: una lista enlazada con un puntero sin inicializar, un archivo abierto que nadie cierra.

¿Cómo hacemos que el tipo mismo impida que esto pase?"

# La solución

struct - SIN CONTROL

```
struct Circulo {
    double radio;  // público
};


Circulo c;
c.radio = -50; // ok
```

Class - CON CONTROL

```
class Circulo {
private:
    double radio;
public:
    void setRadio(double r);
};
// c.radio = -50 da error
```

Una class es una struct que controla quién toca sus datos y garantiza que el objeto siempre esté en un estado válido.

Todo lo que veremos hoy — constructores, destructores, this — son herramientas para hacer cumplir ese contrato.

# Lo que construiremos hoy

Nuestra clase Circulo cumplirá un contrato (lista de capacidades del objeto).

1. El objeto controla el acceso a sus datos:
   → modificadores de acceso, estructura .h / .cpp

2. El objeto nace en estado válido, siempre:
   → constructores y lista de inicialización

3. El objeto se destruye y libera sus recursos: → destructor

4. El objeto sabe a cuál instancia pertenece cada operación → puntero this

5. El objeto rechaza datos inválidos por sí mismo:
   → invariantes de clase, getters con const, setters con validación

# 1. De struct a class: una sola diferencia real

struct – Todo público por defecto

```
struct Circulo {
    double radio;  // público
};


Circulo c;
c.radio = -50; // ok
```

class – Todo privado por defecto

```
class Circulo {
private:
    double radio;
public:
    void setRadio(double r);
};
// c.radio = -50 da error
```

Una clase dice: “mis datos son míos, pide permiso”.

**Ejemplo:** c.radio = -50; no compila porque ahora radio es privado

# Modificadores de acceso: la regla práctica

**private**: **Solo accesible desde dentro de la clase.** Nadie fuera puede leer ni modificar el dato directamente.

**public**: **Accesible desde cualquier lugar.** Ideal para funciones miembro que forman la interfaz del objeto. El exterior no ve los datos, solo los métodos.

**protected**: **Accesible desde la clase y sus clases derivadas.** Lo veremos en detalle cuando lleguemos a herencia.

**REGLA PRÁCTICA**
**Miembros de datos → private**
**Funciones miembro → public**

# Anatomía completa de una clase

```
class Circulo {                 // encabezado: nombre de la clase
private:                        // modificador de acceso
    double radio;               // miembro de dato (atributo)
    double x, y;                // miembro de dato (atributo)


public:                         // modificador de acceso
    double getRadio()  const; // declaración de función miembro - consulta
    double getArea()   const; // declaración de función miembro
    void   setRadio(double r); // declaración de función miembro - modifica


};                              // punto y coma obligatorio
```

- Las funciones en esta sección son solo declaraciones, la definición va en otro lugar.
- El calificador const en getters, le dice al compilador “esta función no modifica el objeto”

# El operador :: y la definición de funciones miembro

```
double Circulo::getRadio() const {
    return radio; // accede al miembro privado directamente
}

double Circulo::getArea() const {
    const double PI = 3.14159;
    return PI * radio * radio;
}

void Circulo::setRadio(double r) {
    radio = r;   // por ahora sin validación – la añadiremos
}
```

- La declaración vive en la clase. La definición vive fuera, calificada con el nombre de la clase y ::
- Dentro de una función miembro los datos privados son visibles (podemos leer y escribir radio directamente).
  El modificador private bloquea el acceso desde afuera de la clase, no desde dentro (encapsulación)
  **Ejemplos** vistos del operador :: std::cout, std::string,

# Tres archivos: estructura de un proyecto real

circulo.h

```h
// circulo.h — DEFINICIÓN DE CLASE (la interfaz)
# pragma once // evita incluir este archivo dos veces

class Circulo {
private:
    double radio;

public:
    double getRadio()  const;
    double getArea()   const;
    void   setRadio(double r);
};
```

circulo.cpp

```cpp
// circulo.cpp — IMPLEMENTACIÓN (el "cómo")
#include "circulo.h"
#include <iostream>
using namespace std;

double Circulo::getRadio() const {
    return radio;
}
double Circulo::getArea() const {
    const double PI = 3.14159;
    return PI * radio * radio;
}
void Circulo::setRadio(double r) {
    radio = r;
}
```

main.cpp

```cpp
// main.cpp — APLICACIÓN (el "qué")
#include "circulo.h"
#include <iostream>
using namespace std;

int main() {
    Circulo c1;
    c1.setRadio(5.0);
    cout << "Radio: " << c1.getRadio() << endl;
    cout << "Área:  " << c1.getArea()  << endl;
    return 0;
}
```

**COMPILACIÓN**

g++ main.cpp circulo.cpp -o programa
./programa

# ¿Cuál compila? ¿Por qué?

Fragmento A

```
int main() {
    Circulo c;
    c.setRadio(3.0);
    cout << c.getRadio();
    return 0;
}
```

Fragmento B

```
int main() {
    Circulo c;
    c.radio = 3.0;
    cout << c.getRadio();
    return 0;
}
```

- **B no compila.** Intenta acceder directamente a radio, que es private. El compilador emite error.

- **A compila**. Usa setRadio(), que es public. El exterior puede llamar funciones miembro públicas.
  A tiene un problema más sutil — Circulo c sin inicializar deja radio con valor basura antes de llamar setRadio. Eso lo resolveremos en el siguiente bloque con constructores.

# 3. El ciclo de vida de un objeto

- Todo objeto nace, trabaja y muere.

**LO QUE VIMOS CON new/ delete**

```
// patrón manual
Circulo* p = new Circulo;
p->radio = 5.0;   // inicializar a mano

// ... operaciones ...

delete p;            // limpiar a mano
```

**LO QUE APRENDEREMOS - automático**

```
// patrón con clase
Circulo c(5.0);    // constructor

// ... operaciones ...

// destructor: automático
// al salir del ámbito
```

**LA PRINCIPAL DIFERENCIA**

- Con new/delete el programador puede olvidar inicializar o liberar.

- Con constructor y destructor, el lenguaje lo garantiza: **siempre se llaman, sin excepción**. Si el objeto existe, fue construido. Si sale del ámbito, fue destruido.

# Constructor de parámetros y lista de inicialización

- El constructor más común. Dos partes: el encabezado con parámetros y la lista de inicialización antes del cuerpo.
-

¿Por qué lista de inicialización?

**EN EL CUERPO – menos eficiente**

```
Circulo::Circulo(double r) {
    radio = r; // asigna después
               // de construir
}
```

**EN LA LISTA – inicializa directamente**

```
Circulo::Circulo(double r)
    : radio(r)  // inicializa
{               // una sola vez

}
```

**IMPORTANCIA:**

Para double la diferencia es mínima. Para miembros de tipo clase (como un string o un objeto anidado) la lista de inicialización evita construir con valor basura y luego sobrescribir.

# Constructor predeterminado y la regla del compilador

- ¿Qué pasa si escribimos Circulo c; sin argumentos? Sin un constructor predeterminado explícito, el compilador se queja.

**LA REGLA DEL COMPILADOR**

- Si defines cualquier constructor, el compilador ya no genera un constructor predeterminado automático.
- Antes de agregar Circulo(double r): Circulo c; funcionaba (el sistema daba uno gratis).
- Después de agregar Circulo(double r) y antes de definir Circulo(): Circulo c; falla con error de compilación.

# El destructor y el orden LIFO

- El destructor se llama automáticamente cuando el objeto sale de su ámbito. El orden en que se destruyen los objetos es inverso al orden de creación (similar a la pila de llamadas).

# Las reglas de los constructores y el destructor

1. El nombre del **constructor** es idéntico al nombre de la clase. Sin valor de retorno — ni siquiera void.

2. Si **defines** cualquier **constructor**, el compilador deja de generar el predeterminado. Si quieres ambos, declara ambos.

3. El **destructor** lleva **~** antes del nombre, sin parámetros y sin valor de retorno. No se puede sobrecargar — solo puede existir uno.

4. El destructor se llama automáticamente al cerrar el ámbito.

5. Si **no defines destructor**, el sistema provee uno vacío. Suficiente cuando no hay recursos dinámicos. Cuando la clase tiene punteros con new, necesitas el definir tu propio destructor.

# Constructor de copia: existe, y el sistema te da uno gratis

- Cuando escribes **Circulo c2 = c1**; o **pasas un Circulo por valor** a una función, C++ **invoca el constructor de copia** – crea un objeto nuevo copiando los datos del objeto fuente. **El sistema provee uno automáticamente**.

# ¿En qué orden se imprimen estos mensajes?

```
void funcion() {
    Circulo x(99.0);
    cout << "dentro de funcion" << endl;
}

int main() {
    Circulo a(1.0);
    funcion();
    Circulo b(2.0);
    return 0;
}
```

- El destructor de x se ejecuta al salir de funcion(), antes de que b sea creado. El objeto vive exactamente mientras vive su ámbito.

# 4. Una sola función 3 objetos distintos

- En memoria solo existe una copia del código de getRadio(). Pero c1.getRadio() devuelve 5 y c2.getRadio() devuelve 10.
  ¿Cómo sabe la función a cuál objeto pertenecer en cada llamada?

**CÓDIGO (UNA SOLA COPIA)** →→

```
getRadio()
getArea()
setRadio()
// puntero oculto: this
```

```
c1  this apunta aquí en t=1
radio = 5.0
0x7ffc1000

c2
radio = 10.0
0x7ffc1008

c3
radio = 3.0
0x7ffc1010
```

**LA RESPUESTA: EL PUNTERO THIS**

C++ agrega automáticamente a cada función miembro un parámetro oculto llamado this. Es un puntero constante al objeto que está invocando la función en ese momento.

# Lo que escribes vs lo que genera el compilador

**LO QUE ESCRIBES**

```
double Circulo::getRadio() const {
    return radio;
}
// llamada:
c1.getRadio();
```

**LO QUE GENERA EL COMPILADOR**

```
double getRadio(const Circulo* this) {
    return this->radio;
}
// llamada traducida:
getRadio(&c1);
```

Cuando **escribes c1.getRadio(),** el compilador traduce eso a **getRadio(&c1) internamente** le pasa la dirección de c1 como el puntero this.

La función entonces accede a this->radio, que es el radio de c1.

# ¿Cuál es el error en este código?

```
class Rectangulo {
private:
    double base;
    double altura;
public:
    Rectangulo(double base, double altura);
    double getArea() const;
};

Rectangulo::Rectangulo(double base, double altura)
    : base(base), altura(altura) { }

double Rectangulo::getArea() {
    return base * altura;
}
```

# 5. Invariantes de clases, diseño defensivo

```
class Circulo {
private:
    double radio;

public:
    Circulo(double r) {
        radio = r;
    }
};
```

¿Es esta clase mejor que struct del inicio?

Aún puedo hacer  Circulo c(-2);

¿Qué es un círculo?

**Un invariante** es una condición que debe ser verdadera durante toda la vida del objeto.
Ejemplo: Circulo, Cuenta Bancaria, Triángulo, Meses del año

**La programación orientada a objetos consiste en encapsular datos y proteger sus invariantes.**

# Constructor defensivo

```
class Circle {
private:
    double radio;


public:
    Circle(double r) {
        if (r > 0)
            radio = r;
        else
            radio = 1;
    }
};
```

Ahora Circle c(-5); produce un objeto válido

Si el constructor falla en proteger el invariante, el objeto nace enfermo. Y un objeto enfermo seguirá propagando errores.

¿Qué sucede con?

```
void setRadio(double r) {
    radio = r;
}
```

Tenemos el mismo problema !

c.setRadio(-100);

**Idea central del diseño defensivo**

**Todo punto de entrada debe proteger el invariante**